

CLASS 10a

RLHF & DPO

Teaching the Model What "Good" Means

From Preference Pairs to Aligned Models

The ChatGPT Origin Story

GPT-3 was released in June 2020.

For 2.5 years, almost nobody outside AI cared.

November 2022: ChatGPT launches.

100 million users in 2 months.

The fastest-growing product in history.

Same base model (GPT-3.5). Same architecture. Same parameters.

What changed?

Four letters: R-L-H-F.

Where We Are

Pre-training

(predict next token)

→ Learns language

CPT

(same loss, domain data)

→ Learns domain language

SFT

(per-token, on responses)

→ Learns to follow instructions

RLHF / DPO

(per-RESPONSE, contrastive)

→ Learns what humans prefer

← **TODAY**

RLVR

[Next session]

→ Learns to reason

Remember What Happened in Class 9b?

Three failures we saw after SFT:

1. SYCOPHANCY

User: "Revenue was \$100B, right?" Context: \$53B

→ Model agrees with wrong information

2. HALLUCINATION

Question about employees, context only has revenue

→ Model: "The company had 47,000 employees." (made up)

3. FORMAT OVERFITTING

Works with template, breaks without it

→ Learned one pattern, not the skill

**SFT couldn't fix these because it only shows GOOD examples.
It never shows BAD examples and says "don't do this."**

The Style Problem

User: "Help, I have to give a presentation tomorrow and I'm panicking."

Response A (blunt):

"You'll be fine. Practice your slides, get some sleep, and stop overthinking it."

Response B (supportive):

"That sounds really stressful. Do one quick practice run, pick the 1-2 points you want people to remember, and give yourself permission to rest tonight."

Both are "correct." Neither is wrong.

But most humans strongly prefer B.

SFT has no mechanism to express "B is better than A."

SFT vs RLHF — The Fundamental Difference

SFT signal:

"Here is a good response. Copy it."

One example at a time.
Per-token. Imitation.

RLHF signal:

"A is BETTER than B."

Two examples compared.
Per-response. Contrastive.

SFT = learning from a textbook (here's the right answer)

RLHF = learning from feedback (this was better than that)

The Two Loss Functions, Side by Side

SFT (per-token):

$$L = - \sum_t \log \pi_{\theta}(y_t \mid x, y_{<t})$$

Sum over every token.
Imitate the training sequence.

DPO (per-response):

$$L = - \log \sigma(\beta \cdot \Delta_{\text{chosen}} - \beta \cdot \Delta_{\text{rejected}})$$

$$\text{where } \Delta(y) = \log [\pi_{\theta}(y|x) / \pi_{\text{ref}}(y|x)]$$

One comparison per response.

**SFT sums over tokens. DPO is one comparison per response.
The structural difference IS the per-token vs per-response shift.**

Why Negative Examples Matter

Teaching someone to draw a circle:

SFT approach:

Show 1000 perfect circles.
"Make it look like this."

Problem: they don't know what
MISTAKES look like.

RLHF approach:

Show pairs:
"This circle > this oval."

Learns the BOUNDARIES
of what's good.

Contrastive learning teaches boundaries.

SFT only teaches the center.

Preferences are inherently about BOUNDARIES — what's acceptable vs not.

SFT Memorizes, RL Generalizes

Proven empirically (Chu et al., 2025):

SFT on Task A: Gets better at Task A. Gets **WORSE** at Task B.

RL on Task A: Gets better at Task A. Task B stays **INTACT**.

Why?

SFT says:

"Your outputs should look EXACTLY like this data."

- ▶ Stretches to match training data
- ▶ Disrupts other capabilities

RL says:

"Find outputs that score well."

- ▶ Improves in one direction
- ▶ Preserves unrelated capabilities

PART 1

REINFORCEMENT LEARNING FOR LLMs

Forget robots. Forget Atari. Think about training a dog.

The RL Objective in One Line

$$\max_{\pi_{\theta}} \quad \mathbb{E}[R(\tau)] - \beta \cdot \text{KL}(\pi_{\theta} \parallel \pi_{\text{ref}})$$

$$\mathbb{E}[R(\tau)]$$

maximize reward

$$\beta \cdot \text{KL}(\pi_{\theta} \parallel \pi_{\text{ref}})$$

don't drift from where you started

Every RL algorithm solves a version of this.

PPO, DPO, GRPO — different math, same goal.

RL — The Dog Training Analogy

You're training a dog to fetch a ball.

You can't EXPLAIN what "fetch" means. (No SFT.)

You can't write down the rules. (No loss function on behavior.)

1. Dog tries something (EXPLORATION)
2. Dog brings back the ball (ACTION)
3. You give it a treat (REWARD)
4. Dog does more of that (POLICY UPDATE)

1. Dog runs the wrong way
2. No treat
3. Dog tries something different

Over time: dog learns "fetch = bring ball = treat."

That's RL. Try → feedback → adjust.
The dog never sees the rules. It discovers them through rewards.

RL Vocabulary Mapped to LLMs

RL Term	Dog Training	LLM
Agent	The dog	The language model
Environment	The park	The conversation
State	Where the ball is	The prompt
Action	Run, fetch, sit	Generate a response
Policy	Dog's learned habits	The model's weights
Reward	Treat (or no treat)	Preference score
Episode	One fetch attempt	One prompt-response pair

The POLICY is what RL optimizes.

For LLMs, the policy IS the model.
Improving the policy = updating the weights
= the model generates better responses.

What IS a Reward? (Concretely)

In robotics RL:

Reward = did the robot reach the target? (+1 or -1) — Clear, objective, automatic.

In LLM RLHF:

Reward = "how good was this response?" But who decides "good"?

Option 1: Ask a human every time

→ Too slow, too expensive

Option 2: Train a MODEL to predict what humans would say

- ▶ THIS is the reward model
- ▶ A learned approximation of human judgment
- ▶ Fast, scalable, but imperfect (it's a proxy)

Bradley-Terry: Pairs → Scores

$$P(\text{chosen} > \text{rejected}) = \sigma(r(\text{chosen}) - r(\text{rejected}))$$

$$\sigma(x) = 1/(1+e^{(-x)}) \quad \text{sigmoid squishes to } [0,1]$$

Reward model loss:

$$L = - \log \sigma(r_{\phi}(\text{chosen}) - r_{\phi}(\text{rejected}))$$

- ▶ Same shape as binary cross-entropy
- ▶ Only DIFFERENCES matter — absolute scale is meaningless

From chess Elo. Pairwise wins → single score per item.

What IS a Policy? (Concretely)

Policy = a strategy for choosing actions given a state

For a chess AI: given this board, which move?

For a robot: given this position, which direction?

For an LLM: given this prompt, which response?

The LLM's policy is literally its weights.

Different weights = different responses to the same prompt.

"Optimizing the policy" = "changing the weights"
= "fine-tuning the model" = what we've been doing all along!

SFT: "make this response more likely" (imitation)

RL: "make high-reward responses more likely" (optimization)

PART 2

THE RLHF PIPELINE

How ChatGPT Was Made

The InstructGPT Recipe (The ChatGPT Recipe)

OpenAI, 2022 — "Training Language Models to Follow Instructions with Human Feedback"

Step 1: SFT [You know this]

Train model to follow instructions

Step 2: Train a REWARD MODEL [NEW today]

Collect human preference pairs (A vs B)

Train a model to score responses

Step 3: Optimize with PPO [NEW today]

Generate → score → update → constrain

Repeat thousands of times

This is the recipe. Every RLHF system follows this template.

The SFT Starting Point

You MUST start from an SFT'd model. Not a base model.

Why?

Base model:

random autocomplete,
no instruction following

After SFT:

follows instructions,
produces structured responses

RLHF needs REASONABLE responses so it can compare them and pick the better one.
If the model generates gibberish, there's nothing to compare.

Pipeline: Base → SFT → RLHF/DPO

Never: Base → RLHF/DPO (won't work)

Step 2 — Collecting Preference Data

For each prompt, generate TWO (or more) responses. A human picks which is better.

Prompt: "Explain what a stock dividend is."

Response A:

"A stock dividend is when a company gives shareholders additional shares instead of cash."

Response B:

"A stock dividend means you get more shares instead of cash. E.g., 5% on 100 shares = 5 extra shares. Total value unchanged — each share just costs less."

Human says: B > A (more complete, has an example)

Thousands of these pairs → training data.
Not about right vs wrong — about BETTER vs WORSE.

What Preference Data Actually Looks Like

Not always obvious good vs bad. Often subtle:

EASY pair (quality gap is obvious):

Chosen: Clear explanation with example

Rejected: Vague, factually wrong, or off-topic

HARD pair (quality gap is subtle):

Chosen:

"Inflation means your money buys less over time. A coffee that cost \$3 last year costs \$3.50 now."

Rejected:

"Inflation is a sustained increase in the general price level of goods and services in an economy."

The HARD pairs teach the MOST. Easy pairs just confirm the obvious.

Step 2b — Training the Reward Model

- ▶ Take an SFT'd language model
- ▶ Remove the language modeling head (next-token prediction)
- ▶ Add a single linear layer that outputs ONE number

```
Input: [prompt + response] → Reward Model → score: 0.73
```

Trained on preference pairs:

```
Given (prompt, chosen, rejected):
```

```
Learn: score(prompt, chosen) > score(prompt, rejected)
```

The model learns which FEATURES make responses better:

Accuracy → **higher score**

Concrete examples → **higher score**

Empathy → **higher score**

Hallucination → **lower score**

Verbosity → **lower score**

All compressed into a single number.

Bradley-Terry — From Chess Rankings to AI

How do you turn "A is better than B" into a loss?

Bradley-Terry model (1952, originally for ranking chess players):

$$P(\text{A preferred over B}) = \text{score(A)} / (\text{score(A)} + \text{score(B)})$$

If A scores 7 and B scores 3:

$$P(\text{A wins}) = 7/10 = 70\%$$

Training: push score(chosen) up, score(rejected) down.

Why not just use "chosen=1, rejected=0"?

Because preferences are PROBABILISTIC.

Even the best response isn't preferred 100% of the time.

Bradley-Terry models this naturally.

Step 3 — PPO Optimization

Now we have:

- ▶ An SFT model (the "policy") that generates responses
- ▶ A reward model that scores responses

PPO (Proximal Policy Optimization) loop:

1. **GENERATE** model produces responses to a batch of prompts
2. **SCORE** reward model scores each response
3. **UPDATE** adjust model to make high-scoring responses more likely
4. **CONSTRAIN** don't drift too far from the SFT model
5. **REPEAT** thousands of times

This is ONLINE training:

The model generates NEW data during training.
It doesn't train on a fixed dataset. It learns from its OWN outputs.

The PPO Objective

What PPO actually maximizes:

$$L_{\text{PPO}} = E[\min(r_t(\theta) \cdot A_t , \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \cdot A_t)]$$

Three pieces, plain English:

$r_t(\theta)$

$$= \pi_{\theta}(a|s) / \pi_{\text{old}}(a|s)$$

"how much more likely under the new model vs the old" = the policy ratio

A_t

= advantage

"how much better was this action than average" = (reward) – (expected reward)

$\text{clip}(r, 1-\epsilon, 1+\epsilon)$

= capping ratio at $[0.8, 1.2]$ for $\epsilon=0.2$

"don't let any single update change probs more than 20%"

PPO = policy gradient, but CLIPPED so you never take a step too big. Clip = the "proximal."

Why PPO is Hard for LLMs

4 models in memory at once:

1. **Active model** (training) $\leftarrow$ gradients, optimizer states

2. **Reference model** (frozen) $\leftarrow$ KL anchor

3. **Reward model** (frozen) $\leftarrow$ scores responses

4. **Value model / critic** $\leftarrow$ predicts expected reward for advantage calc

For a 7B model in fp16:

$4 \times 14 \text{ GB} = 56 \text{ GB}$ just for weights

+ gradients + optimizer states $\approx 150+$ GB total $\rightarrow$ Need an 8xA100 cluster

Plus:

- ▶ Online generation every step (slow)
- ▶ 12+ hyperparameters to tune
- ▶ Training collapses if any are wrong
- ▶ Reward hacking is rampant

This is why DPO was such a relief.

The Total Reward During PPO

$$R_{\text{total}} = R_{\phi}(x, y) - \beta \cdot \text{KL}(\pi_{\theta} || \pi_{\text{ref}})$$

$R_{\phi}(x, y)$

reward model

$\beta \cdot \text{KL}(\pi_{\theta} || \pi_{\text{ref}})$

leash to reference

β **small** → loose leash → faster learning, risk of collapse

β **large** → tight leash → less drift, slower learning

$\beta = 0$ → no leash → guaranteed reward hacking

Typical: $\beta = 0.01$ to 0.1

This subtraction is the entire defense against reward hacking.

The Reference Model — What and Why

During RLHF, keep a FROZEN copy of the SFT model = the "reference model."

Why?

- ▶ The reference is your safety net
- ▶ It defines "what the model was like before RLHF"
- ▶ The KL penalty says: "You can improve, but stay CLOSE to the reference"

```
Final_reward = RM_score - beta * KL(policy || reference)
```

beta is the leash.

Too loose (low beta):

model wanders, reward hacks.

Too tight (high beta):

model barely changes.

Goodhart's Law — The Fundamental Problem

**"When a measure becomes a target,
it ceases to be a good measure."**

The reward model is a PROXY for human preferences. Optimize too hard:

Model learns: "Adding 'I hope this helps!' gets +0.3 reward"

→ Adds it to EVERY response

Model learns: "Longer responses score higher"

→ Generates walls of text for simple questions

Model learns: "Confident language scores higher"

→ Becomes confidently wrong instead of honestly uncertain

This is REWARD HACKING.

The KL penalty, beta tuning, and early stopping fight this.

Why RLHF Is Hard and Expensive

FOUR models in GPU memory simultaneously:

1. **Policy model** (being trained)

2. **Reference model** (frozen SFT copy, for KL)

3. **Reward model** (scores generations)

4. **Value model** (estimates future reward for PPO)

For 70B: $4 \times 140 \text{ GB} = 560+ \text{ GB VRAM minimum}$

Plus:

- ▶ Human labeling: \$100K - \$1M
- ▶ Online training: must GENERATE during training (slow)
- ▶ Instability: reward hacking, mode collapse
- ▶ Hyperparameter hell: beta, LR, PPO epochs, KL target

This is why for 2 years only OpenAI and Anthropic did RLHF. Everyone else: SFT.

PART 3

DPO — THE SIMPLIFICATION

"What if we could skip the reward model entirely?"

The DPO Insight

Rafailov et al., NeurIPS 2023: "Direct Preference Optimization"

Key insight:

The optimal policy under RLHF can be expressed DIRECTLY in terms of the policy and reference model. No reward model needed.

RLHF: preferences → reward model → RL → better model

DPO: preferences → better model (one step!)

Models in memory: 4 → 2 (policy + reference only)

Online generation: not needed (works on fixed dataset)

Human labeling: same data as RLHF

Stability: much easier (it's basically SFT with a new loss)

How DPO Works — The Intuition

For each training example: `(prompt, chosen_response, rejected_response)`

DPO asks two questions:

- ▶ How much does the POLICY like chosen vs rejected?
- ▶ How much does the REFERENCE like chosen vs rejected?

If the policy already prefers chosen (matching the reference):

→ Small update. Already aligned.

If the policy prefers rejected (disagreeing with the data):

→ Large update. Fix this.

The "relative to reference" part IS the KL constraint.
It's baked into the loss function, not a separate penalty.

The DPO Loss, Decoded

$$L_{\text{DPO}} = -\log \sigma \left(\beta \cdot \log \left[\frac{\pi_{\theta}(y_c|x)}{\pi_{\text{ref}}(y_c|x)} \right] - \beta \cdot \log \left[\frac{\pi_{\theta}(y_r|x)}{\pi_{\text{ref}}(y_r|x)} \right] \right)$$

Read it inside-out:

$$\log \left[\frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)} \right]$$

"how much MORE likely to generate y now compared to where we started" = IMPLICIT REWARD (no reward model needed!)

$$\text{chosen_reward} - \text{rejected_reward}$$

= preference margin

$$\sigma(\text{margin})$$

→ probability of correct preference

$$-\log(\dots)$$

→ standard cross-entropy

Why Contrasting Works Better Than Imitation

SFT approach:

- ▶ Train on CHOSEN only
- ▶ Ignore rejected
- ▶ Works OK, misses half the signal

DPO approach:

- ▶ Push chosen UP
- ▶ Push rejected DOWN
- ▶ Learns both — what to do AND what to avoid

The rejected response contains INFORMATION:

- ▶ "This phrasing is too verbose"
- ▶ "This tone is too blunt"
- ▶ "This answer is too vague"

Throwing away the rejected = throwing away lessons.

DPO uses the FULL signal. SFT uses half.

DPO Is Offline — A Key Difference from RLHF

RLHF (PPO): ONLINE training

- ▶ Model generates DURING training
- ▶ New data every step
- ▶ More powerful but expensive and unstable

DPO: OFFLINE training

- ▶ Works on a FIXED dataset
- ▶ Same data every epoch
- ▶ Simpler, cheaper, more stable

Trade-off:

Online: model can explore and discover new good behaviors

Offline: model can only learn from what's in the dataset

For most use cases, offline is enough.

For frontier performance, online still wins slightly.

DPO vs RLHF — Side by Side

	RLHF (PPO)	DPO
Models in memory	4	2
Training	Online (generates)	Offline (fixed dataset)
Reward model	Required	Not needed
Stability	Tricky	Stable (like SFT)
Compute	Very expensive	Moderate
Human data	Preference pairs	Same pairs!
Performance	Best at frontier	Very close (90-95%)
Learning rate	$\sim 1e-6$	$\sim 5e-7$ to $5e-6$

DPO trades a small performance gap for massive simplicity.

This is why DPO exploded in 2024.

The Learning Rate Breakthrough

THE single most important DPO hyperparameter:

SFT learning rate:

$2e-4$

DPO learning rate:

$5e-7$ to $5e-6$

(10-100x LOWER!)

Early DPO papers (2023): "DPO doesn't work that well."

They used SFT-scale learning rates.

Zephyr team (late 2023): "Try 10-100x lower."

Suddenly DPO works beautifully.

Why? DPO loss is more sensitive than SFT loss.

SFT: "predict this token" (gentle signal)

DPO: "prefer THIS over THAT" (strong contrastive signal)

High LR overshoots. Low LR lets it settle.

Why DPO LR Must Be 10–100× Lower (the math)

DPO gradient hits HARDER than SFT per step:

- ▶ Each update affects BOTH chosen AND rejected probs
- ▶ Both measured relative to reference (amplified)
- ▶ Net update $\approx 2\times$ what SFT would produce

Plus the policy/reference ratio is exponential —
small weight changes $\rightarrow$ large probability swings.

Practical:

SFT $\rightarrow 2e-4$
DPO $\rightarrow 5e-6$ to $5e-7$ (40–400× lower)

- ▶ If loss bounces $\rightarrow$ halve LR.
- ▶ If loss is flat $\rightarrow$ double LR.
- ▶ Never above $1e-5$ unless you know what you're doing.

The Preference Dataset

DPO needs: (prompt, chosen_response, rejected_response)

Popular datasets:

UltraFeedback — 64K pairs, diverse tasks (we use this today)

Nectar — 183K pairs, multi-turn

HH-RLHF — Anthropic's helpfulness/harmlessness data

```
{
  "prompt": "Explain inflation in simple terms.",
  "chosen": "Inflation means prices go up over time,
            so your money buys less. A $3 coffee
            this year might cost $3.50 next year.",
  "rejected": "Inflation is a sustained increase in
              the general price level of goods and
              services in an economy..."
}
```

Both correct. Chosen is simpler, uses a concrete example.

PART 4

HANDS-ON DPO TRAINING

The Plan

- 1 See why SFT can't distinguish quality (demo)
- 2 Load UltraFeedback preference dataset
- 3 Train with DPO (TRL's DPOTrainer + LoRA)
- 4 Understand DPO metrics (rewards/margins/accuracy)
- 5 Test: does the model prefer chosen over rejected?
- 6 Test for over-optimization (length bias)

Rejection Sampling — The Underrated Middle Ground

Even simpler than DPO:

1. Generate 10-30 responses per prompt
2. Score them with a reward model
3. Keep only the best ones
4. SFT on the best responses

No RL. No contrastive loss. Just "generate, filter, retrain."

Used by: Llama 3, Tulu 3, most production systems.

Often combined with DPO in multi-stage pipelines.

Think of it as: curating your own SFT dataset using a scorer.

DPO Variant Landscape

DPO spawned a family of "Direct Alignment Algorithms":

DPO (2023) — the original, most tested

IPO (2024) — fixes overconfidence

KTO (2024) — works with thumbs-up/down (no pairs needed!)

ORPO (2024) — combines SFT + DPO in one step

SimPO (2024) — no reference model needed

Start with DPO. Most documented and supported.

Branch out only for specific needs.

When to Use What — The Complete Cheat Sheet

Symptom	Fix
Model doesn't know the domain	CPT
Model doesn't follow instructions	SFT
Model's answers lack style/quality	DPO
Model agrees when it shouldn't	DPO + pushback pairs
Model hallucinates confidently	DPO + refusal pairs
Model needs to reason better	RLVR (next session)

Production pipeline:

Base → CPT → SFT → Rejection Sampling → DPO → (RLVR)

Each step fixes what the previous couldn't.

The One Equation to Remember

$$L_{\text{DPO}} = -\log \sigma(\beta \cdot \hat{\Delta}_{\text{chosen}} - \beta \cdot \hat{\Delta}_{\text{rejected}})$$

$$\hat{\Delta}(y) = \log \left[\pi_{\theta}(y|x) / \pi_{\text{ref}}(y|x) \right]$$

Three knobs:

- β preference strength (0.1)
- LR** step size (5e-6)
- data** preference pair quality (matters most)

Three failure modes:

- Reward hacking** → check length, formatting
- Overfitting** → use eval set, early stop
- Drift** → β too low OR LR too high

Key Takeaways

1. SFT shows good examples. RLHF/DPO shows good vs bad.
2. SFT memorizes, RL generalizes. (Proven, not theoretical.)
3. RLHF pipeline: SFT model → reward model → PPO. 4 models.
4. DPO: same result, no reward model, 2 models, offline.
5. You MUST start from an SFT'd model. Never DPO a base model.
6. Reward hacking / Goodhart's Law is the fundamental challenge.
7. Rejection sampling is underrated — generate, filter, retrain.
8. DPO learning rate: $5e-7$ to $5e-6$ (10-100x lower than SFT!)